

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №8.

Тема: «Программа, управляемая событиями»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Базовый класс: ПЕЧАТНОЕ ИЗДАНИЕ (PRINT)

Название – string

Автор – string

Производный класс: ЖУРНАЛ (MAGAZINE)

Количество страниц – int

Группа – Список (List).

Команды:

Создать группу (формат команды: m количество элементов группы).

Добавить элемент в группу (формат команды: +)

Удалить элемент из группы (формат команды: -)

Вывести информацию об элементах группы (формат команды: s)

Вывести информацию о названиях всех элементов группы (формат команды: z)

Конец работы (формат команды: q)

Анализ задачи

1. Создаём абстрактный класс PrintEdition с защищёнными полями name и author. Добавляем конструкторы, виртуальный деструктор, чисто виртуальные методы input и output, геттеры и сеттеры, перегрузку оператора присваивания, виртуальный метод getInfo.

2. Создаём производный класс Magazine от PrintEdition. Добавляем поле pages. Создаём конструкторы, деструктор, конструктор копирования, перегрузку оператора присваивания, геттеры и сеттеры для pages. Переопределяем методы input, output и getInfo.

3. Создаём класс Group для хранения списка указателей на Magazine. Используем list<Magazine*> для хранения. Создаём конструкторы, деструктор, конструктор копирования, перегрузку оператора присваивания. Создаём метод addElement для добавления журнала. Создаём метод removeElement для удаления по названию. Создаём метод viewGroup для вывода всех журналов. Создаём метод viewNames для вывода названий. Создаём метод getSize для получения размера. Создаём метод clear для очистки всех элементов.

4. Создаём производный класс Dialog от Group. Добавляем поле running для управления циклом работы. Создаём приватные методы handleCreate (создание группы из N элементов), handleAdd (добавление элемента), handleRemove (удаление элемента), handleView (просмотр группы), handleViewNames (просмотр названий). Создаём виртуальные методы-события onElementAdded, onElementRemoving, onGroupViewed для демонстрации полиморфизма. Создаём метод run для запуска интерактивного диалога с пользователем (команды m, +, -, s, z, q, h). Создаём метод showHelp для вывода справки.

5. В функции main: тестируем класс Magazine (создание объектов, копирование, присваивание, изменение полей). Тестируем класс Group (добавление, удаление, просмотр). Запускаем диалоговую систему Dialog для интерактивной работы с пользователем.

Код

```
#include <iostream>
#include <string>
#include <list>
#include <algorithm>
#include <clocale>

using namespace std;

class PrintEdition {
protected:
    string name;
    string author;

public:
    PrintEdition() : name(""), author("") {}
    PrintEdition(const string& n, const string& a) : name(n), author(a) {}
    virtual ~PrintEdition() {}

    virtual void input() = 0;
    virtual void output() const = 0;

    string getName() const { return name; }
    string getAuthor() const { return author; }

    void setName(const string& n) { name = n; }
    void setAuthor(const string& a) { author = a; }

    PrintEdition& operator=(const PrintEdition& other) {
        if (this != &other) {
            name = other.name;
            author = other.author;
        }
        return *this;
    }

    virtual string getInfo() const {
        return "Название: " + name + ", Автор: " + author;
    }
};

class Magazine : public PrintEdition {
private:
    int pages;

public:
    Magazine() : PrintEdition(), pages(0) {}
    Magazine(const string& n, const string& a, int p) : PrintEdition(n, a), pages(p) {}
    ~Magazine() {}

    Magazine(const Magazine& other) : PrintEdition(other), pages(other.pages) {}

    Magazine& operator=(const Magazine& other) {
        if (this != &other) {
            PrintEdition::operator=(other);
            pages = other.pages;
        }
        return *this;
    }

    int getPages() const { return pages; }
    void setPages(int p) { pages = p; }
```

```

void input() override {
    cout << "Введите название журнала: ";
    cin.ignore();
    getline(cin, name);
    cout << "Введите автора: ";
    getline(cin, author);
    cout << "Введите количество страниц: ";
    cin >> pages;
}

void output() const override {
    cout << "Журнал: " << name << ", Автор: " << author
        << ", Страниц: " << pages << endl;
}

string getInfo() const override {
    return "Журнал: " + name + ", Автор: " + author + ", Страниц: " +
to_string(pages);
}
};

class Group {
protected:
    list<Magazine*> items;

public:
    Group() {}

    Group(const Group& other) {
        for (auto* mag : other.items) {
            items.push_back(new Magazine(*mag));
        }
    }

    virtual ~Group() {
        clear();
    }

    Group& operator=(const Group& other) {
        if (this != &other) {
            clear();
            for (auto* mag : other.items) {
                items.push_back(new Magazine(*mag));
            }
        }
        return *this;
    }

    void addElement(Magazine* mag) {
        items.push_back(mag);
        cout << "Элемент добавлен. Текущий размер: " << getSize() << endl;
    }

    bool removeElement(const string& name) {
        auto it = find_if(items.begin(), items.end(),
            [&name](Magazine* mag) { return mag->getName() == name; });

        if (it != items.end()) {
            delete* it;
            items.erase(it);
            cout << "Элемент удален. Текущий размер: " << getSize() << endl;
            return true;
        }
        cout << "Элемент с названием \"" << name << "\" не найден!" << endl;
        return false;
    }

    virtual void viewGroup() const {

```

```

        if (items.empty()) {
            cout << "Группа пуста!" << endl;
            return;
        }
        cout << "\nСодержимое группы (размер: " << getSize() << ")" << endl;
        int i = 1;
        for (auto* mag : items) {
            cout << i++ << ". ";
            mag->output();
        }
    }

    void viewNames() const {
        if (items.empty()) {
            cout << "Группа пуста!" << endl;
            return;
        }
        cout << "\nНазвания журналов в группе:" << endl;
        for (auto* mag : items) {
            cout << " - " << mag->getName() << endl;
        }
    }

    int getSize() const {
        return items.size();
    }

    void clear() {
        for (auto* mag : items) {
            delete mag;
        }
        items.clear();
    }

    list<Magazine*>& getItems() { return items; }
    const list<Magazine*>& getItems() const { return items; }
};

class Dialog : public Group {
private:
    bool running;

    void handleCreate(int count) {
        cout << "\nСоздание группы из " << count << " элементов" << endl;
        clear();
        for (int i = 0; i < count; i++) {
            cout << "\nЭлемент " << i + 1 << ":" << endl;
            auto* mag = new Magazine();
            mag->input();
            items.push_back(mag);
        }
        cout << "Группа создана! Размер: " << getSize() << endl;
    }

    void handleAdd() {
        cout << "\nДобавление элемента" << endl;
        auto* mag = new Magazine();
        mag->input();
        addElement(mag);
        onElementAdded(*mag);
    }

    void handleRemove() {
        cout << "\nУдаление элемента" << endl;
        if (items.empty()) {
            cout << "Группа пуста, нечего удалять!" << endl;
            return;
        }
        viewNames();
    }
};

```

```

    string name;
    cout << "Введите название журнала для удаления: ";
    cin.ignore();
    getline(cin, name);

    auto it = find_if(items.begin(), items.end(),
        [&name](Magazine* mag) { return mag->getName() == name; });
    if (it != items.end()) {
        onElementRemoving(*it);
    }
    removeElement(name);
}

void handleView() {
    viewGroup();
    onGroupViewed();
}

void handleViewNames() {
    viewNames();
}

virtual void onElementAdded(const Magazine& mag) {
    cout << "[Событие] Добавлен элемент: " << mag.getInfo() << endl;
}

virtual void onElementRemoving(const Magazine& mag) {
    cout << "[Событие] Удаляется элемент: " << mag.getInfo() << endl;
}

virtual void onGroupViewed() {
    cout << "[Событие] Выполнен просмотр группы" << endl;
}

public:
    Dialog() : running(true) {}

    void run() {
        string command;
        cout << "\nСИСТЕМА УПРАВЛЕНИЯ ГРУППОЙ ЖУРНАЛОВ" << endl;
        showHelp();

        while (running) {
            cout << "\n> ";
            cin >> command;

            if (command == "m") {
                int count;
                cin >> count;
                if (count > 0) {
                    handleCreate(count);
                }
                else {
                    cout << "Количество должно быть положительным!" << endl;
                }
            }
            else if (command == "+") {
                handleAdd();
            }
            else if (command == "-") {
                handleRemove();
            }
            else if (command == "s") {
                handleView();
            }
            else if (command == "z") {
                handleViewNames();
            }
            else if (command == "q") {

```

```

        cout << "Завершение работы..." << endl;
        running = false;
    }
    else if (command == "h") {
        showHelp();
    }
    else {
        cout << "Неизвестная команда! Введите 'h' для справки." << endl;
    }
}

void showHelp() {
    cout << "\nДоступные команды:" << endl;
    cout << "  m <количество>  - создать группу из N элементов" << endl;
    cout << "  +                - добавить элемент в группу" << endl;
    cout << "  -                - удалить элемент из группы" << endl;
    cout << "  s                - вывести информацию об элементах группы" << endl;
    cout << "  z                - вывести названия всех элементов" << endl;
    cout << "  q                - завершить работу" << endl;
    cout << "  h                - показать эту справку" << endl;
}

};

int main() {
    setlocale(LC_ALL, "Russian");

    cout << "ТЕСТИРОВАНИЕ ИЕРАРХИИ КЛАССОВ" << endl;

    Magazine mag1("Наука и жизнь", "Иванов А.А.", 128);
    Magazine mag2("Вокруг света", "Петров Б.Б.", 96);

    cout << "\nОбъект 1: ";
    mag1.output();
    cout << "Объект 2: ";
    mag2.output();

    Magazine mag3(mag1);
    Magazine mag4;
    mag4 = mag2;

    cout << "\nКопия mag1: ";
    mag3.output();
    cout << "Присвоенный mag2: ";
    mag4.output();

    cout << "\nНазвание mag1: " << mag1.getName() << endl;
    cout << "Автор mag1: " << mag1.getAuthor() << endl;
    mag1.setName("Новое название");
    mag1.setAuthor("Новый автор");
    cout << "После изменения: ";
    mag1.output();

    cout << "\nРабота с группой" << endl;
    Group group;
    group.addElement(new Magazine("Tech Today", "Smith J.", 64));
    group.addElement(new Magazine("Health & Life", "Brown M.", 48));
    group.viewGroup();
    group.viewNames();
    cout << "Размер группы: " << group.getSize() << endl;

    group.removeElement("Tech Today");
    group.viewGroup();

    cout << "\nТекущий размер: " << group.getSize() << endl;

    cout << "\nЗапуск диалоговой системы" << endl;

    Dialog dialog;

```

```

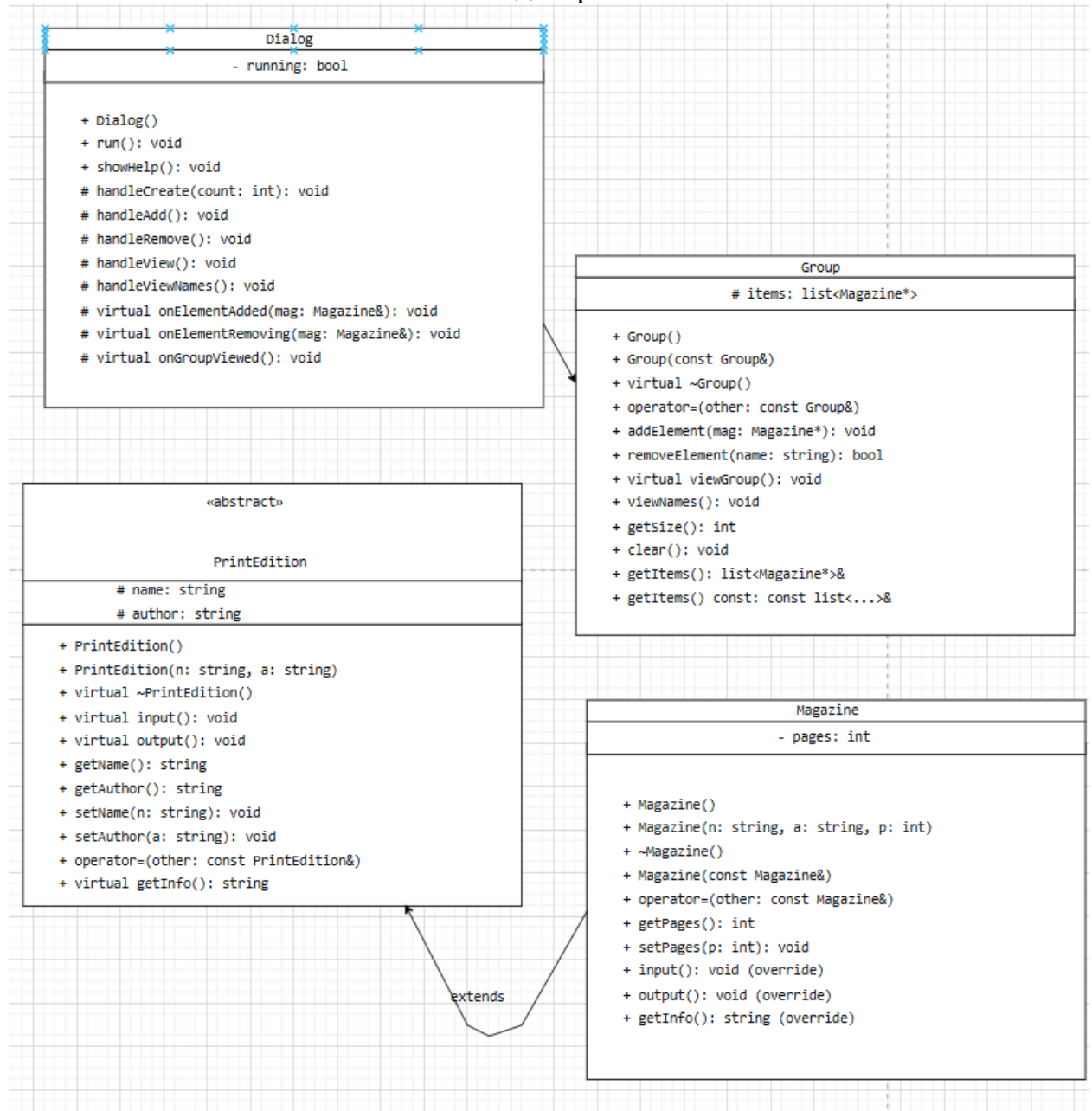
dialog.run();

cout << "\nПРОГРАММА ЗАВЕРШЕНА" << endl;

return 0;
}

```

UML диаграмма



Вывод программы


```

C:\Users\palerin\source\repos\ConsoleApplication14\bin\Debug\ConsoleApplication14.exe
ТЕСТИРОВАНИЕ ИЕРАРХИИ КЛАССОВ

Объект 1: Журнал: Наука и жизнь, Автор: Иванов А.А., Страниц: 128
Объект 2: Журнал: Вокруг света, Автор: Петров Б.Б., Страниц: 96

Копия mag1: Журнал: Наука и жизнь, Автор: Иванов А.А., Страниц: 128
Присвоенный mag2: Журнал: Вокруг света, Автор: Петров Б.Б., Страниц: 96

Название mag1: Наука и жизнь
Автор mag1: Иванов А.А.
После изменения: Журнал: Новое название, Автор: Новый автор, Страниц: 128

Работа с группой
Элемент добавлен. Текущий размер: 1
Элемент добавлен. Текущий размер: 2

Содержимое группы (размер: 2)
1. Журнал: Tech Today, Автор: Smith J., Страниц: 64
2. Журнал: Health & Life, Автор: Brown M., Страниц: 48

Названия журналов в группе:
- Tech Today
- Health & Life
Размер группы: 2
Элемент удален. Текущий размер: 1

Содержимое группы (размер: 1)
1. Журнал: Health & Life, Автор: Brown M., Страниц: 48

Текущий размер: 1

Запуск диалоговой системы

СИСТЕМА УПРАВЛЕНИЯ ГРУППОЙ ЖУРНАЛОВ

Доступные команды:
m <количество> - создать группу из N элементов
+               - добавить элемент в группу
-               - удалить элемент из группы
s               - вывести информацию об элементах группы
z               - вывести названия всех элементов
q               - завершить работу
h               - показать эту справку

> h

Доступные команды:
m <количество> - создать группу из N элементов
+               - добавить элемент в группу
-               - удалить элемент из группы
s               - вывести информацию об элементах группы
z               - вывести названия всех элементов
q               - завершить работу
h               - показать эту справку

> m 2

Создание группы из 2 элементов

Элемент 1:
Введите название журнала: Наука и жизнь
Введите автора: Иванов В.В.
Введите количество страниц: 128

Элемент 2:
Введите название журнала: Вокруг света
Введите автора: Петров Б.Б.
Введите количество страниц:
96
Группа создана! Размер: 2

```

Ответы на вопросы

1. Класс-группа — класс, предназначенный для хранения и управления группой однородных объектов. Примеры: список, массив, очередь, стек, множество.

2. Описание класса-группы Список (List) в C++:

```

class List
{
private:
    struct Node
    {
        int data;

```

```

        Node* next;
    };
    Node* head;
public:
    List();
    List(const List& other);
    ~List();
    void add(int value);
    void show();
};

```

3. Конструкторы для класса List:

```

List::List() : head(nullptr) { }

```

```

List::List(int value) : head(nullptr)
{
    add(value);
}

```

```

List::List(const List& other) : head(nullptr)
{
    Node* current = other.head;
    while (current)
    {
        add(current->data);
        current = current->next;
    }
}

```

4. Деструктор для класса List:

```

List::~~List()
{
    while (head)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

```

5. Метод для просмотра элементов класса List:

```

void List::show()

```

```

{
    Node* current = head;
    while (current)
    {
        cout << current->data << " ";
        current = current->next;
    }
}

```

6. Группа даёт иерархию контейнеров — древовидную структуру, где базовые классы определяют интерфейс, а производные — конкретные способы хранения.

7. Абстрактный класс должен находиться во главе иерархии, потому что он задаёт единый интерфейс для всех классов группы, запрещает создание бессмысленных объектов базового типа и обеспечивает полиморфизм.

8. Событие — это уведомление о наступлении некоторого действия (нажатие клавиши, движение мыши, таймер). События используются для обработки взаимодействия с пользователем и для передачи сообщений между объектами.

9. Характеристики события-сообщения: тип события (what), время возникновения, источник события, данные, связанные с событием (координаты, код клавиши).

10. Пример структуры, описывающей событие:

```

struct MouseEventArgs
{
    int x;
    int y;
    int button;
};

```

11. Полю what присваиваются значения, идентифицирующие тип события. Например: 1 для события мыши, 2 для нажатия клавиши, 3 для сообщения. Конкретные значения выбирает разработчик.

12. Полю command присваиваются значения, идентифицирующие команду. Например: 1 для команды открыть файл, 2 для сохранить, 3 для выхода. Конкретные значения зависят от логики приложения.

13. Поля a и message используются для передачи дополнительных параметров команды. Например: message — идентификатор сообщения, a — числовой параметр команды.

14. Для организации обработки сообщений необходимы методы: `GetEvent` (получить событие), `HandleEvent` (обработать событие), `ClearEvent` (очистить очередь), `Valid` (проверить корректность).

15. Главный цикл обработки событий-сообщений имеет вид:

```
while (true)
{
    TEvent event;
    GetEvent(event);
    if (event.what == evQuit) break;
    HandleEvent(event);
    ClearEvent(event);
}
```

16. Метод `ClearEvent()` удаляет обработанное событие из очереди. Реализуется перемещением указателя на следующее событие или удалением элемента из контейнера.

17. Метод `HandleEvent()` обрабатывает событие в зависимости от его типа (`what`). Реализуется через `switch` или полиморфные вызовы.

18. Метод `GetEvent()` извлекает следующее событие из очереди событий. Он проверяет наличие событий и возвращает первое необработанное.

19. Поле `EndState` используется для определения состояния завершения (например, выход из приложения). Это поле обычно содержится в классе приложения или главного окна.

20. Функция `Valid()` проверяет корректность состояния объекта или введенных данных перед выполнением операции. Возвращает `true`, если все условия соблюдены, и `false` в противном случае.